

PHASE 2

Requirement Analysis Phase

EduGenie - AI Powered Learning Assistant

Convert the project idea into clear functional, non-functional, technical, and user-interface requirements.

Prepared as a phase-wise academic submission document based on the supplied EduGenie-Ai project archive.

1. Functional Requirements

FR-01	Home page	Serve the main EduGenie web interface at /.
FR-02	Health check	Return service status from /health.
FR-03	Question answering	Accept text input and return an AI-generated answer from /qa.
FR-04	Explanation	Accept a topic and return an AI-generated explanation from /explain.
FR-05	Quiz generation	Accept a topic/passage and return quiz data from /quiz.
FR-06	Summarization	Accept text and return a concise summary from /summarize.
FR-07	Learning recommendations	Accept a topic and learner level and return a learning path from /learn/recommendations.
FR-08	Result presentation	Display generated text or quiz content in the browser.
FR-09	Client validation	Prevent empty submissions and show status/error messages.
FR-10	Copy result	Allow the visible result to be copied using the browser clipboard API.

2. Non-Functional Requirements

- Usability: simple task selection, input field, generate action, clear action, and result area.
- Performance: avoid unnecessary client-side processing and return API responses asynchronously.
- Reliability: backend exceptions are converted into HTTP 502 responses with error details.
- Maintainability: learning tasks are separated into Python modules and exposed through distinct endpoints.
- Security: API credentials should be supplied through environment variables and must not be hard-coded into documentation or source control.
- Responsive UI: CSS includes a mobile breakpoint for narrow screens.

3. Technical Requirements Evidenced in Archive

Backend	FastAPI
Server	Uvicorn
Templating	Jinja2
Configuration	python-dotenv / .env
AI provider SDK	google-genai
Validation / models	Pydantic
HTTP client	httpx
Testing dependency	pytest
Frontend	HTML template + CSS + vanilla JavaScript
Configured AI model	gemini-2.5-flash

4. Input and Output Requirements

The JavaScript sends JSON with a text field to each learning endpoint. For the learning-path task, it also sends a level field. Text results are read from a result property. Quiz handling expects a result object containing a questions collection and question fields such as question, options, and correct_answer.

5. Constraints and Risks

- AI output can vary between requests and should be validated before user display.
- External API access requires a valid Gemini key and network connectivity.
- The supplied archive is not a complete runnable snapshot because referenced Python modules are absent and the HTML template file is empty.
- The JavaScript references an element with id level-container and other UI elements that depend on the HTML template being populated.

6. Requirement Acceptance Criteria

A requirement is considered satisfied when the related UI control, backend route, request payload, response model, and visible output can be exercised successfully in an environment with the complete project source and valid API configuration.